Arquitecturas Dinámicas LEMP mediante DOCKER COMPOSE

En el desarrollo de aplicaciones modernas, es fundamental recrear entornos de producción de forma fiel en las máquinas locales.

La arquitectura **LEMP** (Linux, Nginx, MySQL y PHP) es una de las más utilizadas por su alto rendimiento y eficiencia.

Mediante el uso de **Docker Compose**, los desarrolladores pueden definir en un único manifiesto YAML el comportamiento de múltiples contenedores, sus redes internas y sus dependencias, garantizando que el servidor web, el motor de procesamiento y la base de datos funcionen de manera integrada y aislada.

- **Diseñar una arquitectura multi-contenedor** integrada que separe las responsabilidades de servicio web, procesamiento de aplicaciones y persistencia de datos.
- **Gestionar la comunicación interna** entre contenedores mediante nombres de servicio y redes bridge.
- **Configurar entornos dinámicos** mediante el uso de variables de entorno para la inicialización de bases de datos MySQL.
- **Implementar pruebas de conectividad** entre servicios para validar el despliegue de la capa de negocio y la capa de datos.

Requisitos del Entorno

- Sistema operativo con **Docker Engine** y el plugin de **Docker Compose** instalado (Linux, Windows con WSL2 o macOS).
- Editor de código profesional (ej: VS Code).
- Cliente de base de datos o acceso a terminal para validación de logs.

Tareas a Realizar

Fase 1: Diseño del Manifiesto de Orquestación

El alumno deberá crear un archivo docker-compose.yml que orqueste tres servicios específicos:

1. **Nginx:** Encargado de la recepción de tráfico HTTP y el servicio de archivos estáticos. Debe mapear el puerto **8085 del host** al puerto **80 del contenedor** y depender del servicio PHP.
2. **PHP-FPM:** Procesador dedicado para la lógica de la aplicación PHP (imagen php:fpm).
3. **MySQL:** Motor de base de datos (versión 5.7 o superior). Se deben definir variables de entorno para:
    - MYSQL_ROOT_PASSWORD: Contraseña administrativa.
    - MYSQL_DATABASE: Nombre de la base de datos a crear inicialmente.

Fase 2: Desarrollo del Código de Aplicación y Conectividad

En el directorio raíz del proyecto (que estará vinculado mediante volúmenes a los contenedores de Nginx y PHP), crea un archivo index.php avanzado que realice las siguientes acciones:

- Muestre la configuración del entorno PHP (phpinfo).
- Implemente un script de conexión a la base de datos MySQL utilizando el nombre del servicio definido en el YAML como **hostname**.
- Imprima un mensaje de éxito si la conexión es satisfactoria o un error descriptivo en caso de fallo, validando así la comunicación inter-contenedor.

Fase 3: Despliegue y Monitorización

1. **Levantamiento:** Inicia la pila completa en segundo plano utilizando docker compose up -d.
2. **Verificación de Estado:** Utiliza docker ps para asegurar que los tres servicios están en estado *Running*.
3. **Inspección de Logs:** Analiza la salida de los registros de MySQL y PHP-FPM con docker compose logs para confirmar la correcta inicialización del sistema y la creación de la base de datos.

Fase 4: Validación Final

- Accede desde el navegador host a http://localhost:8085.
- Confirma que se visualiza el contenido dinámico generado por PHP y que la conexión a la base de datos MySQL aparece como "Exitosa".

Instrucciones de Entrega

Se debe entregar un informe técnico en formato **PDF** con la nomenclatura **DAW_UD01_DockerLEMP_NombreApellido.pdf**.

El documento debe incluir de forma obligatoria:

- **Contenido completo del archivo** docker-compose.yml, debidamente formateado e indentado.
- **Captura de pantalla de la terminal** ejecutando los comandos docker compose up y docker ps.
- **Captura del navegador** donde se aprecie la URL y el resultado del script PHP de conexión a MySQL.
- **Descripción técnica:** Explica brevemente por qué es necesario que Nginx "dependa de" (depends_on) PHP para que el entorno funcione correctamente.
- **Análisis de problemas:** Breve resumen de cualquier conflicto (puertos ocupados, errores de sintaxis en PHP, etc.) y cómo se resolvió.

Nota: Se valorará especialmente la limpieza en el código PHP y el uso de buenas prácticas en la definición de las variables de entorno.